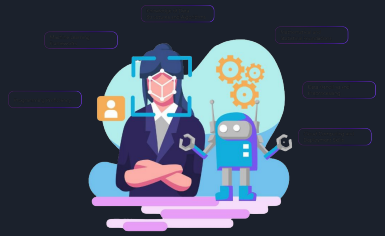
Welcome to AI engineer course

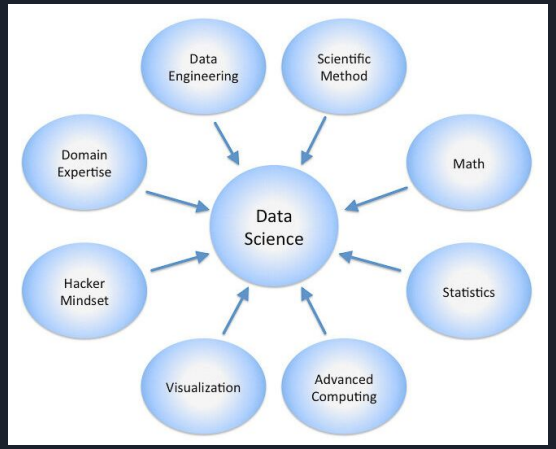
Lecture agenda

- What is Data Science?
- Who is AI engineer?
- Why Python for AI engineering?
- Work areas
  - Jupyter
  - Google Colab
  - GPU access
- Python essentials
  - Python Syntax & Semantics
  - Functions, Modules, and Packages
  - Virtual Environments and Dependency Management (Pip, Poetry, Conda)
  - Basic File I/O and OS Operations
  - Exception Handling
  - Intro to OOP in Python



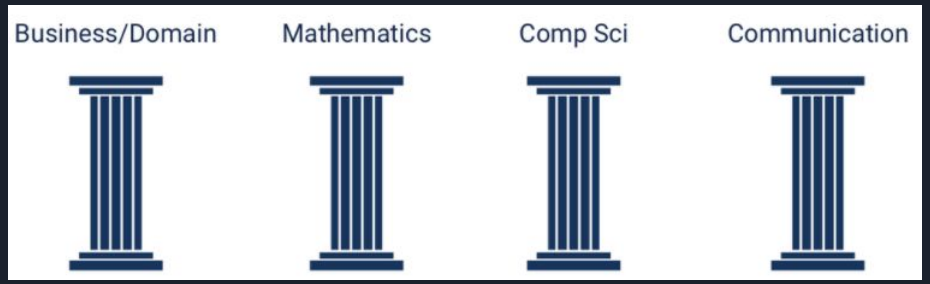
What is data science

- By the Harvard University data scientist is the most promising job of the 21st Century.
- The definition of the role can be confused by the fact that there are other roles sometimes thought of as the same, but often quite different. Some of these include data analysts, data engineers, and so on.
- Here is a diagram showing some of the common disciplines that a data scientist may draw upon.



The pillars of Data Science expertise

- An ideal case a data scientist will be an expert in four fundamental areas. In no particular order of priority or importance.

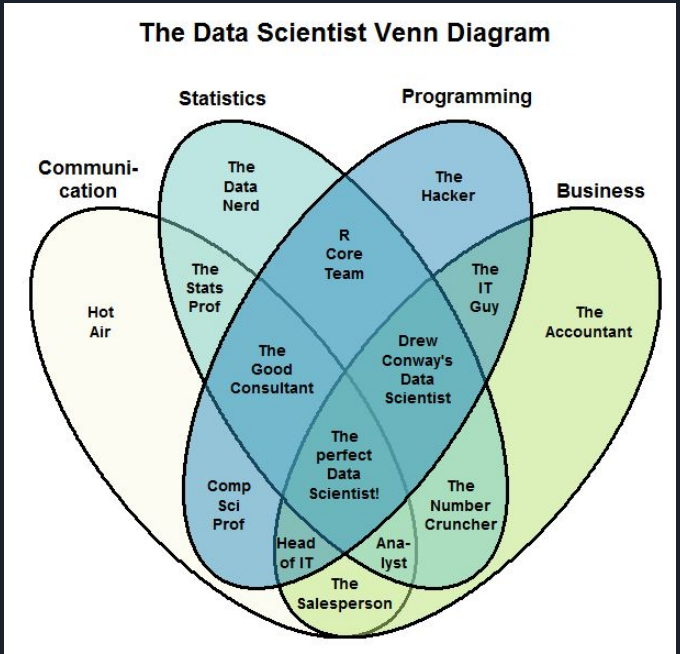


- In reality a good data scientist will be a good one or two of these pillars, but usually not equally strong in all four.

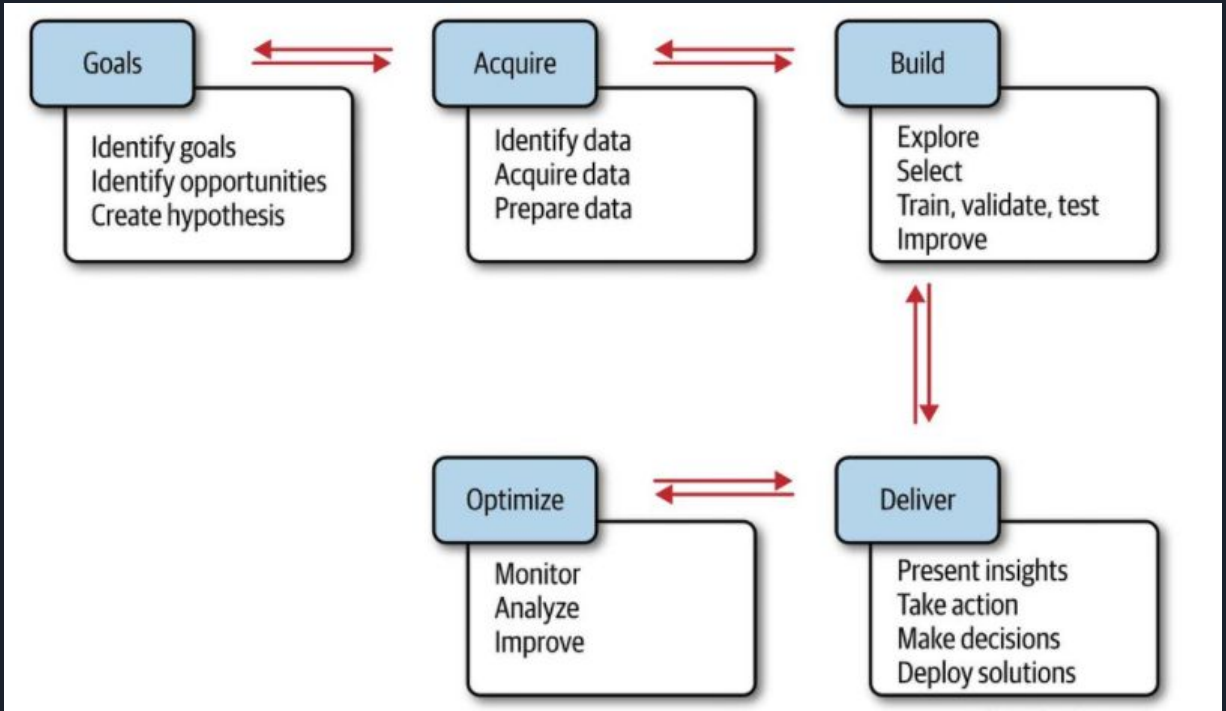
- Based on these pillars, my data scientist definition is a person who should be able to leverage existing data sources, and create new ones as needed to extract meaningful information and actionable sights.

The Data scientist Venn Diagram

This diagram, and others like it, attempt to assign labels and/or characterize the person or field that lies at the intersection of each of the primary competencies shown.



The Data Science Process



Who is AI engineer?

Definition:

An AI Engineer is a specialist who combines software engineering, data science, and machine learning expertise to design, develop, and deploy intelligent systems that turn data into actionable solutions.

Key Responsibilities:

- **Model Development:** Research, select, and train machine learning and deep learning models.
- **Data Engineering:** Build and maintain pipelines to collect, clean, and transform data.
- **Software Integration:** Embed AI components into applications and services using best coding practices.
- **Deployment & Scaling:** Containerize, deploy, and monitor models in production environments.
- **Evaluation & Optimization:** Continuously test performance, refine algorithms, and ensure ethical, reliable outcomes.

Why Python for AI Engineering?

- **Rich Ecosystem:**
  Extensive libraries (TensorFlow, PyTorch, scikit-learn) and tools (NumPy, Pandas, Matplotlib) streamline model building and data processing.
- **Ease of Learning & Readability:**
  Clean, English-like syntax accelerates development and collaboration across teams.
- **Rapid Prototyping:**
  Interactive environments (Jupyter Notebooks) enable quick experimentation and iterative tuning.
- **Strong Community & Support:**
  Vast open-source community, abundant tutorials, and active forums ensure fast troubleshooting and continuous innovation.
- **Seamless Integration:**
  Easy to embed AI modules into web services (Flask, Django) or deploy at scale with Docker and Kubernetes.
- **Cross-Platform Compatibility:**
  Runs on Windows, macOS, and Linux, facilitating consistent development and production environments.

Python brief history

- Python (since 1991) is a programming language that is easy to learn and very powerful. Thus, Python is an ideal language for rapid application development.

- Python 2.0 was released on October 16, 2000, with many significant new features, including a cycle-detecting garbage collector (in addition to reference counting) for memory management and support for Unicode. However, the most important change was the development process , shifting to a more transparent and community-backed process.

- Python 3.0, a major, backward-incompatible release, was released on December 3, 2008, after a long testing period. Many of its major features have also been backported to the backward-compatible, though now-unsupported, Python 2.6 and 2.7.

Guido van Rossum

Anaconda

What Is Anaconda?

A free, open-source Python distribution tailored for data science and machine learning.

Core Components:

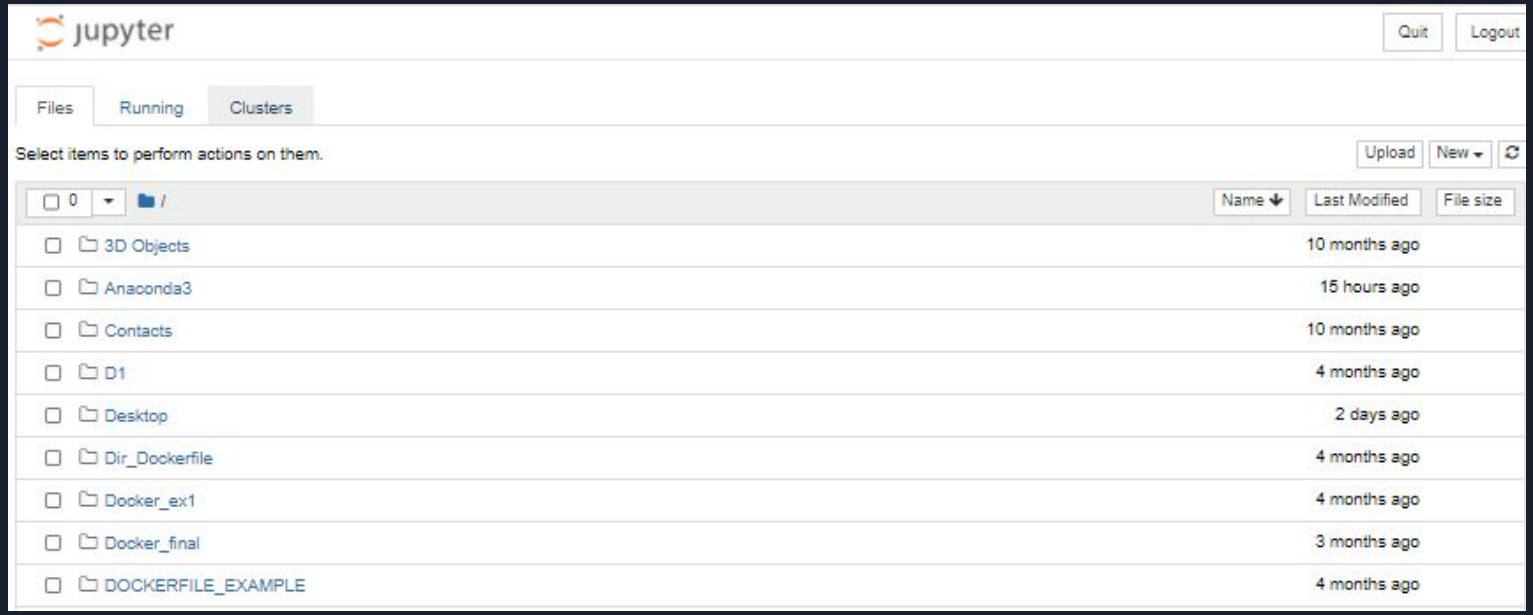
- **Conda Package Manager:** Installs, updates, and removes libraries with a single command.
- **Environment Isolation:** Create separate Python environments to avoid dependency conflicts.
- **Pre-Bundled Libraries:** Comes with 150+ data science packages (NumPy, Pandas, SciPy, Jupyter)

Why Use Anaconda?

- **Reproducibility:** Share exact environments via environment.yml.
- **Ease of Setup:** One installer for Python, packages, and tools.
- **Seamless Integration:** Built-in support for Jupyter Notebooks and IDEs.

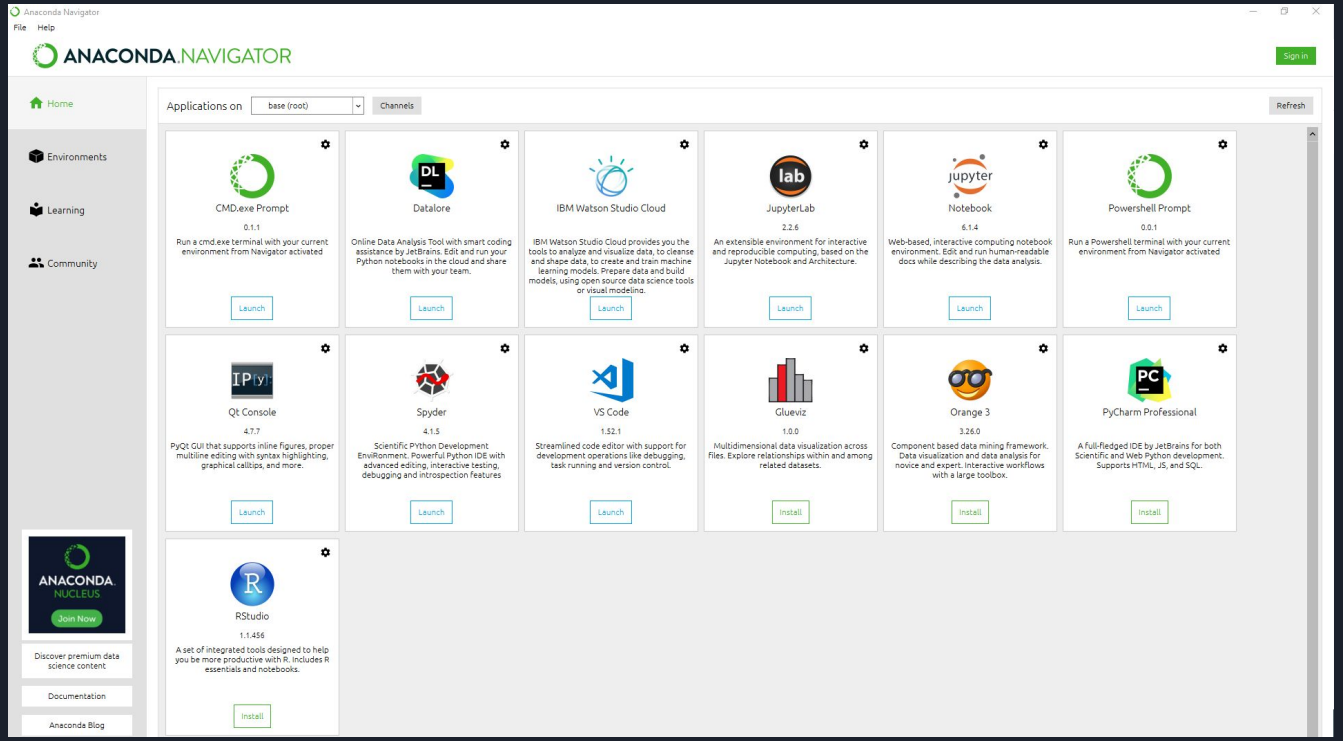
Jupyter notebook access

```
(base) C:\Users\alexk>jupyter notebook
```



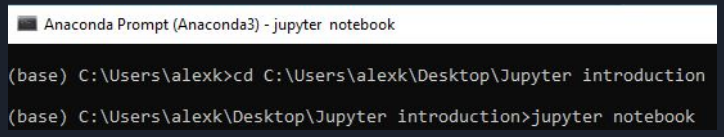
Anaconda navigator

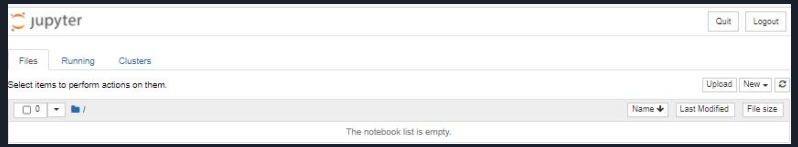
In Anaconda navigator we will see the tools that are coming with the Anaconda installation.



Jupyter notebook

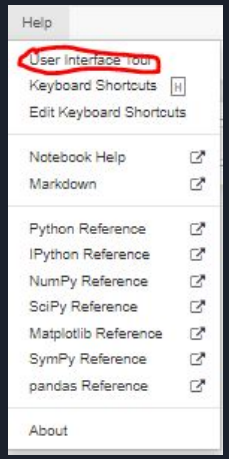
- The Jupyter Notebook is a web application that allows you to create and share documents that contain live code, equations, visualizations, and explanatory text. Uses include data cleaning and transformation, numerical simulation, statistical modeling, machine learning, and much more.
- Let’s start to work with the Jupyter notebook and see some cool examples.
- The first thing that we will do is to create a new directory on your desktop and open the Jupyter notebook from there.

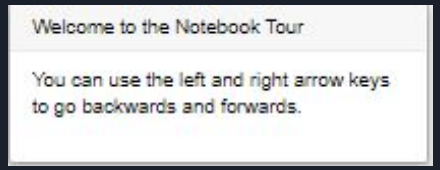


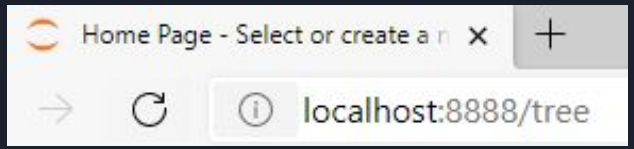


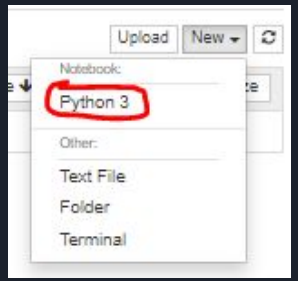
Jupyter notebook continue

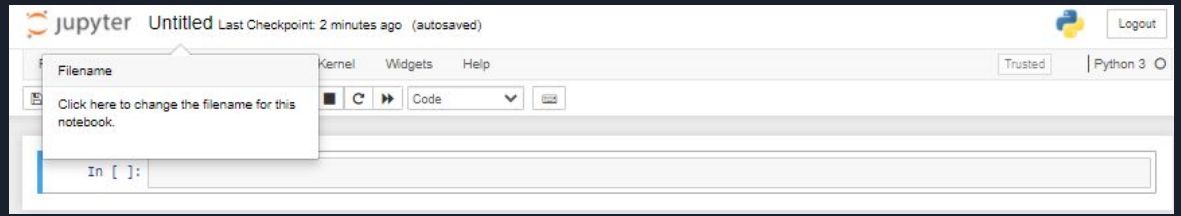
- We can see that the Jupyter notebook server is placed under our localhost server, port 8888.
- Because The Jupyer is running on a web server we must leave the running terminal during our whole work time, otherwise, the server will stop its workflow.
- Let’s create a new notebook in our working directory.
- Let’s make a short user interface tour.







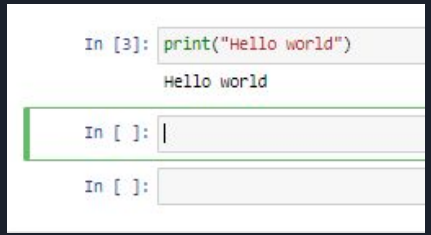




Jupyter notebook continue

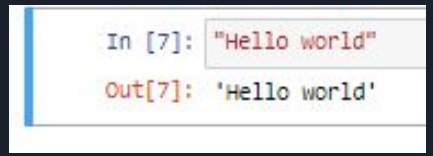
- We see the output of the execution on the screen and the cell that we just ran is still selected.
- Let's see what happens when we choose "Run cells and select bellow". We will see that we ran the code and selected the cell below.

- And the last option that we will see will be "Run cells and insert bellow".

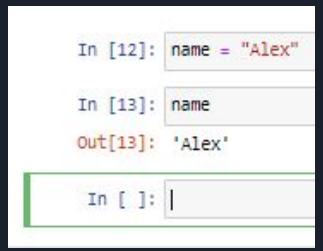


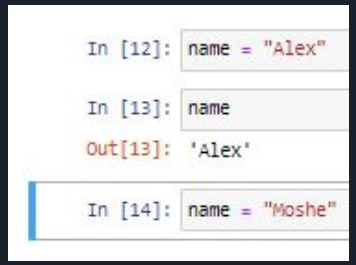
Jupyter notebook continue

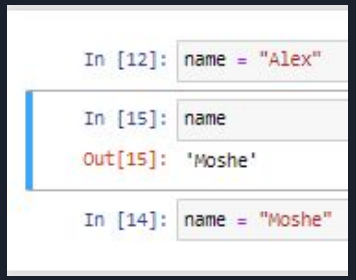
- To remove the cells we need to select the wanted cell and click on “x”.
- We will use the provided shortcuts to run the cells, so our workflow will be faster.
- Remember that the Jupyer notebook is working in interactive mode, this means that we can perform the commands similar to the Python console.



- As you can see we have numbers near the lines, these are the numbering of the executed cells. Let’s see an example to understand it better.

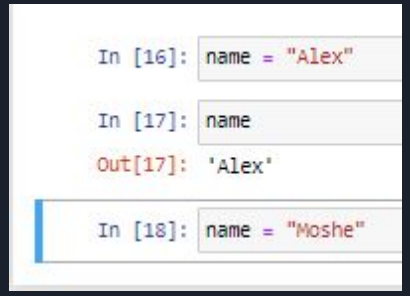




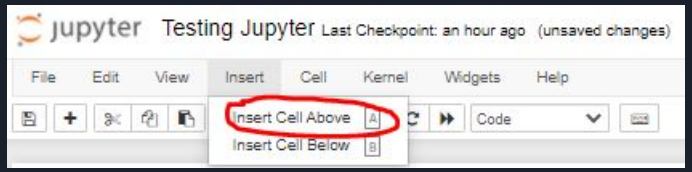


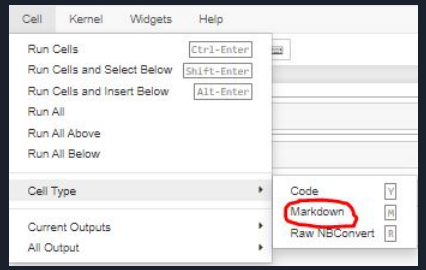
Jupyter notebook continue

- In case that you want to run all cells from top to bottom you can just use the Run all command.

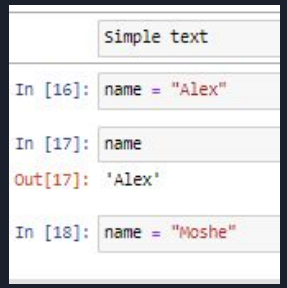


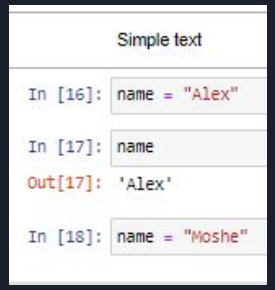
- We can declare our cell not just as a code cell but as a Markdown cell as well. In this cell, we can present information as simple text or we can use HTML syntax for the required presentation.

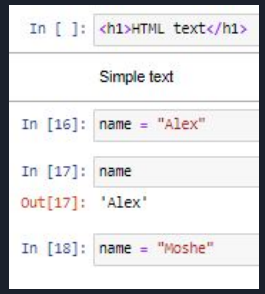


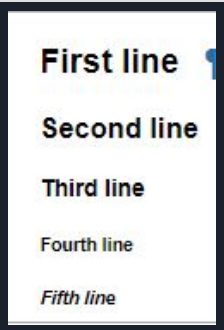


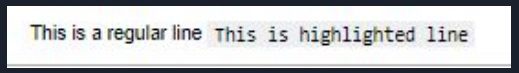
Jupyter notebook continue











We can declare our cell not just as a code cell but as a Markdown cell as well. In these cells, we can present information as simple text or we can use HTML syntax for the required presentation. Also, we have additional shortcuts for presentations.

Jupyter notebook continue

```
__Bold text__
**Same as Bold**
_Italic text_
*Same as Italic*
***Bold and Italic***
```

Bold text Same as Bold *Italic text Same as Italic* ***Bold and Italic***

To go jump down between the lines we have two main options. The first is to add two spaces after each line. The other way is to add
 between the lines.

```
__Bold text__
**Same as Bold**
_Italic text_
*Same as Italic*
***Bold and Italic***
```

Bold text
Same as Bold
Italic text
Same as Italic
Bold and Italic

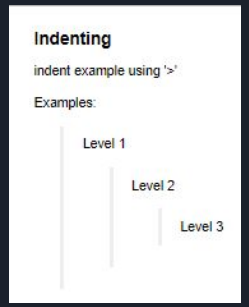
```
__Bold text__
<br>
**Same as Bold**
<br>
_Italic text_
<br>
*Same as Italic*
<br>
***Bold and Italic***
```

Bold text
Same as Bold
Italic text
Same as Italic
Bold and Italic

Indenting

indent example using '>'

Examples:
> Level 1
>> Level 2
>>> Level 3




```
### Coloring
<font color = green> This is a green line. </font>
<br>
<font color = blue> This is a blue line. </font>
```

Jupyter notebook continue

```
### Bullets and Numbering
This is my bullet
- My name is Alex
1. I love Python
2. I love Jupyter
```

Bullets and Numbering

This is my bullet

- My name is Alex

1. I love Python
2. I love Jupyter

```
### Equations

$(\frac{a_1}{a_2} + \frac{a_3}{a_4})^2 = a_5^3$
```

Equations

$$(\frac{a_1}{a_2} + \frac{a_3}{a_4})^2 = a_5^3$$

Jupyter notebook continue

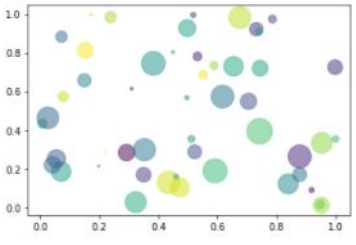
- Let’s see the available line magic's. Let’s run some Linux commands.

```
In [21]: %lsmagic
Out[21]: Available line magics:
%alias  %alias_magic  %autoawait  %autocall  %automagic  %autosave  %bookmark  %cd  %clear  %cls  %colors  %conda  %config  %connect_info  %copy  %ddir  %debug  %dhist  %dirs  %doctest_mode  %echo  %ed  %edit  %env  %gui  %hist  %history  %killbgscripts  %ldir  %less  %load  %load_ext  %loadpy  %logoff  %logon  %logstart  %logstate  %logstop  %ls  %lsmagic  %macro  %magic  %matplotlib  %mkdir  %more  %notebook  %page  %pastebin  %pdb  %pdef  %pdoc  %pfile  %pinfo  %pinfo2  %pip  %popd  %pprint  %precision  %prun  %psearch  %psource  %pushd  %pwd  %pycat  %pylab  %qtconsole  %quickref  %recall  %rehashx  %reload_ext  %ren  %rep  %rerun  %reset  %reset_selective  %rmdir  %run  %save  %sc  %set_env  %store  %sx  %system  %tb  %time  %timeit  %unalias  %unload_ext  %who  %who_ls  %whos  %xdel  %xmode

Available cell magics:
%%!  %%HTML  %%SVG  %%bash  %%capture  %%cmd  %%debug  %%file  %%html  %%javascript  %%js  %%latex  %%markdown  %%perl  %%prun  %%pypy  %%python  %%python2  %%python3  %%ruby  %%script  %%sh  %%svg  %%sx  %%system  %%time  %%timeit  %%writefile

Automagic is ON, % prefix IS NOT needed for line magics.

In [22]: %pwd
Out[22]: 'C:\\Users\\alexk\\Desktop\\Jupyter introduction'

In [ ]:
```

```
In [24]: %ls
 Volume in drive C has no label.
 Volume Serial Number is 842A-0DC6

 Directory of C:\Users\alexk\Desktop\Jupyter introduction

03/27/2021  08:56 PM    <DIR>          .
03/27/2021  08:56 PM    <DIR>          ..
03/27/2021  02:03 PM    <DIR>          .ipynb_checkpoints
03/27/2021  08:56 PM             7,177 Testing Jupyter.ipynb
               1 File(s)          7,177 bytes
               3 Dir(s)  33,878,822,912 bytes free

In [ ]:
```

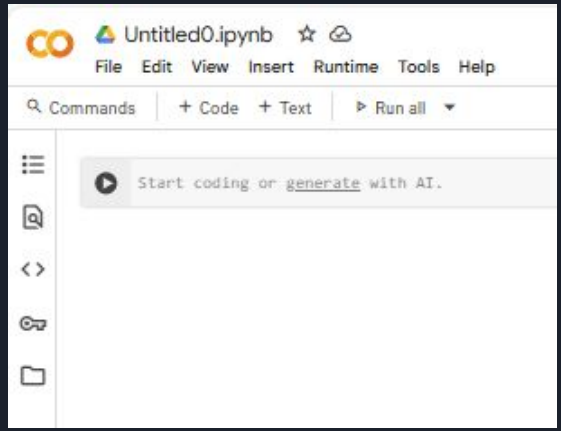
- Another cool thing is the presentation of the plots. Let’s try to run a basic code that is presented in the matplotlib site.

```
In [26]: import numpy as np
         import matplotlib.pyplot as plt
         N = 50
         x = np.random.rand(N)
         y = np.random.rand(N)
         colors = np.random.rand(N)
         area = np.pi * (15 * np.random.rand(N)) ** 2
         plt.scatter(x, y, s=area, c=colors, alpha=0.5)
         plt.show()
```



Google Colab

- **What It Is:**
  A free, cloud-hosted Jupyter notebook service by Google.
- **Zero Setup:**
  Start coding immediately—no local installation or configuration needed.
- **Compute Resources:**
  Access free CPUs, GPUs, and TPUs to accelerate model training.
- **Seamless Collaboration:**
  Share notebooks via link; collaborators can view and edit in real time.
- **Built-In Integrations:**
  Mount Google Drive, import from GitHub, and install libraries with !pip or !apt.
- **Ideal For:**
  Rapid prototyping, data exploration, teaching, and sharing reproducible experiments.



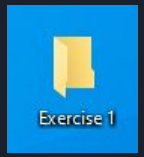
GPU

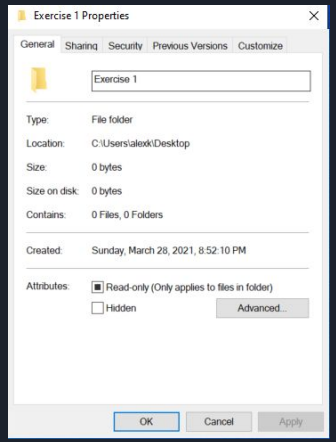
- **Parallel Speed-up:** Hundreds of cores accelerate matrix/tensor ops, cutting training time by 10–100× vs. CPU.
- **Easy Access via Colab:** Enable “Runtime → Change runtime type → GPU” in Google Colab for free, cloud-hosted GPU.
- **Framework Support:** NVIDIA CUDA & cuDNN under the hood of TensorFlow, PyTorch, JAX.

Exercise

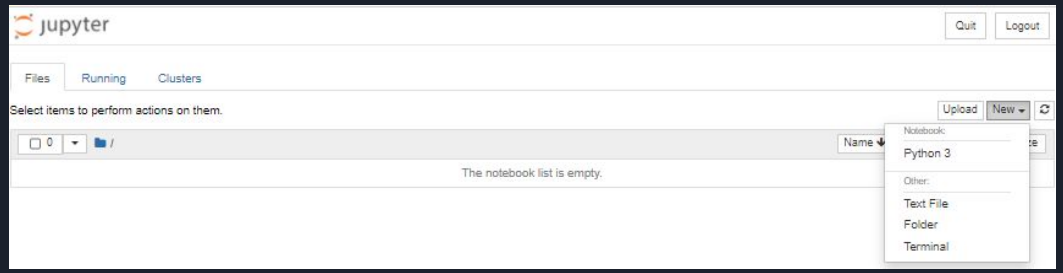
- Make sure that you installed Conda and you have access to Jupyter notebook. Make sure that you are able to access Google Colab.
- Create a new directory and open a new Jupyter notebook inside the directory.
- Create three Markdown cells. The first one will present three titles in descending size. The second one will present two lines, the first line will present your name in bold, the second line will present your phone number in italic, each line will be more indented to the previous lines. The third one will present two lines, the first line your name in red color, the second line your phone number in pink color.
- Create a cell that will run Python print command that will print your home address.
- Change the name of your notebook and save it. Make sure that you see the structure of your final notebook in JSON format.

Solution part 1






```
(base) C:\Users\alexk>cd "C:\Users\alexk\Desktop\Exercise 1"

(base) C:\Users\alexk\Desktop\Exercise 1>jupyter notebook
```




```
# Title 1
## Title 2
### Title 3
```

Title 1

Title 2

Title 3

```
__Alex Kuznetsov__
<br>
_0527389001_
```

Alex Kuznetsov
0527389001

Solution part 2

```
<font color = red> Alex Kuznetsov </font>
<br>
<font color = pink> 0527389001 </font>
```

Alex Kuznetsov
0527389001

```
In [1]: print("Mendes 69")
        Mendes 69

In [ ]:
```

jupyter Exercise1

Rename Notebook

Enter a new notebook name:

Exercise1

Cancel Rename

```
{
 "cells": [
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "# Title 1  \n",
    "## Title 2  \n",
    "### Title 3"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
    "__Alex Kuznetsov__\n",
    "<br>\n",
    "_0527389001_"
   ]
  },
  {
   "cell_type": "markdown",
   "metadata": {},
   "source": [
```

Data types in Python

Data types in Python:

- Numeric : int, float, complex
- Text : str
- Boolean : bool
- Sequence : list, tuple, range
- Set : set, frozentset
- Mapping : dict
- Binary : bytes, bytearray, memoryview

```
In [2]: firstvar = "Hello my dear friends"
        print(firstvar)

        Hello my dear friends

In [3]: firstvar = 2
        print(firstvar)

        2
```

- Variables declaration.

```
In [5]: print("The address:", id(firstvar))
        print("The type of the value:", type(firstvar))
        type("Alex")

        The address: 140711012607808
        The type of the value: <class 'int'>

Out[5]: str
```

- We have a built-in function in Python that allows us to see the address that the variable points to and we have a function that allows us to see the type of the value.

Arithmetic operators

- Arithmetic operators examples.

```
In [6]: num1 = 10
        num2 = 6
        print(num1 + num2)
        print(num1 - num2)
        print(num1 * num2)
        print(num1 / num2)
        print(num1 // num2)
        print(num1 % num2)
        print(num1 ** 2)
        type(num1)

        16
        4
        60
        1.6666666666666667
        1
        4
        100

Out[6]: int
```

```
In [7]: #Arithmetic actions order in Python
        num3 = (num1 + num2) * 5
        print(num3)

        80
```

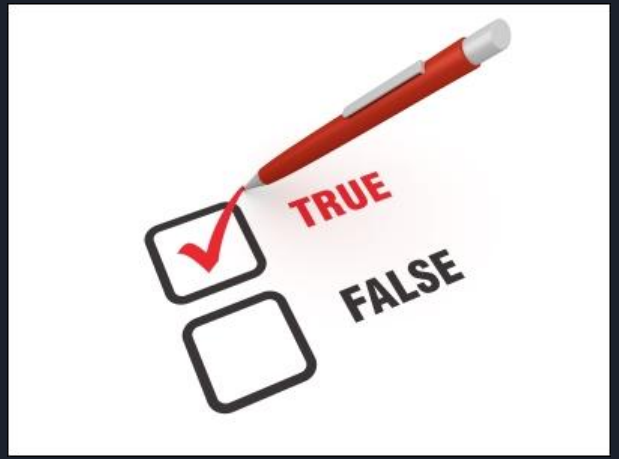
- Arithmetic operators orders in Python.
- We have shortcuts for the usage of the variable if we are activating an operation of the variable on itself.

```
In [9]: # Increment and decrement of numeric
        # Example
        num = 9
        num = num + 1
        print("Example:", num)
        #Shortcuts exmaples
        num += 1
        print("Incremental shortcut:", num)
        decnum = 9
        decnum -= 1
        print("Decremental shortcut:", decnum)

        Example: 10
        Incremental shortcut: 11
        Decremental shortcut: 8
```

Comparison Operators

In [17]:
```
#Boolean examples:
positive_val = True
negative_val = False
print(f"Positive value type {type(positive_val)}")
print(f"Negative value type {type(negative_val)}")

#Comparison operators
print("1. ", 5.0 == 5)
print("2. ", 6 != 2*3)
print("3. ", -2 >= 1)
print("4. ", 3 <= 3)
x = 3 < 3
print("5. ", x)

#Comparison operators on characters
print("6. ", 'a' != 'b')
print("7. ", 'a' < 'b')
print("8. ", 'A' < 'a')
```

```
Positive value type <class 'bool'>
Negative value type <class 'bool'>
1.  True
2.  False
3.  False
4.  True
5.  False
6.  True
7.  True
8.  True
```



Logical Operators

In [19]:
```
#Logical operators
#And examples:
print("And 1. ", True and False)
print("And 2. ", True and True)
print("And 3. ", False and False)
#Or examples:
a = True
b = False
print("Or 1. ", a or b)
print("Or 2. ", a or True)
print("or 3. ", b or 3 < 6)
#Not examples:
print("Not 1. ", not a)
print("Not 2. ", not "A" > "a")
```

```
And 1.  False
And 2.  True
And 3.  False
Or 1.  True
Or 2.  True
or 3.  True
Not 1.  False
Not 2.  True
```

Strings methods, functions and operators

```python
#Strings methods ,functions and operators
str_example = "Hello world"
print(f"The type of the value {type(str_example)}")
#Length function
print(f"The length of str_example: {len(str_example)}")
#Operators between string values:
x = str_example + " Alex"
print(f"We can use + operator between two strings, the result: {x}")
x *= 3
print(f"We can use * operator between two strings, the result: {x}")
#Which other arithmetical operators we can use between the strings?
#upper and lower cases
print(f"We can use the upper case method: {str_example.upper()}")
print(f"We can use the lower case method: {str_example.lower()}")
```

```
The type of the value <class 'str'>
The length of str_example: 11
We can use + operator between two strings, the result: Hello world Alex
We can use * operator between two strings, the result: Hello world AlexHello world AlexHello world Alex
We can use the upper case method: HELLO WORLD
We can use the lower case method: hello world
```

Strings methods, functions and operators continue

```
#rstrip example
space_example = "My name is Alex        "
another_str = "Hello"
print("Before:", space_example + another_str)
rstrip_example = space_example.rstrip()
print("After:", rstrip_example + another_str)
```

```
Before: My name is Alex        Hello
After: My name is AlexHello
```

```
#lstrip example
space_example = "        My name is Alex"
print("Bfore:", space_example)
lstrip_example = space_example.lstrip()
print("After:", lstrip_example)
```

```
Bfore:         My name is Alex
After: My name is Alex
```

```
#We can also use the strip methods on specific values
str_example = "Hello world"
rstrip_specific = str_example.rstrip("ld")
print(rstrip_specific)
```

```
Hello wor
```

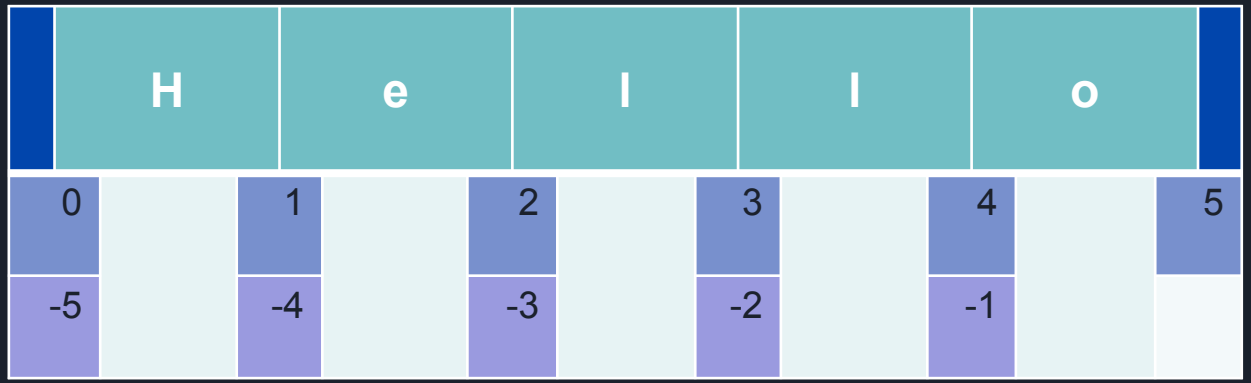
```
#strip method
space_example = "    My name is Alex    "
strip_example = space_example.strip()
print("Before:", space_example)
print("After:", strip_example)
```

```
Before:     My name is Alex
After: My name is Alex
```

```
#Both methods combination
long_str = "**********Alex#############"
both_example = long_str.lstrip("*").rstrip("#")
print("Before:", long_str)
print("After:", both_example)
```

```
Before: **********Alex#############
After: Alex
```

index Access




```python
#Strings access examples
a = "Hello"
print("The value at index 1:", a[1])
print("The value from index 1 to index 3:", a[1:3])
print("The value from index 1 to end of the string:", a[1:])
print("The value from index -4 to index -2:", a[1:3])
print("The value from the beginning to index -3:", a[:-3])
print("The value from index 1 to end of the string:", a[-3:])
```

```
The value at index 1: e
The value from index 1 to index 3: el
The value from index 1 to end of the string: ello
The value from index -4 to index -2: el
The value from the beginning to index -3: He
The value from index 1 to end of the string: llo
```

Strings Access continue

```
#Strings access edge cases
a = "Hello"
"What will happen if i will try to access this?"
print(a[2:1])
"What will happen if i will try to access this?"
print(a[1:7193739123])
#What will happen if i will try to access this?
print(a[7193739123])
```

```
ello

---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-15-94fb700393af> in <module>
      6 print(a[1:7193739123])
      7 #What will happen if i will try to access this?
----> 8 print(a[7193739123])

IndexError: string index out of range
```

```
#Strings access jump declaration
a = "Hello"
print(a[:4:2])
print(a[::-1])
```

```
Hl
olleH
```

Exercise

- Create an string variable named x that is equal to “*****#### Hello World   &&& $$$$”.
- Clean the string from all special characters with the relevant methods. Save the cleaned value in a variable named y.
- Create a new variable z that will hold the lower case of the string that is placed under y, do it with a relevant method.
- Create a new string ALL that will hold the length of z + y string.
- Print x,y,z, and ALL.

Solution

```
x = "*****####  Hello World    &&& $$$$"
y = x.lstrip("*").lstrip("#").lstrip().rstrip("$").rstrip().rstrip("&").rstrip()
z = y.lower()
ALL = len(z + y)
print("x:", x)
print("y:", y)
print("z:", z)
print("ALL:", ALL)
```

```
x: *****####  Hello World    &&& $$$$
y: Hello World
z: hello world
ALL: 22
```

Converting one data type to another

We can convert a value to be from a different data type if there are no logical contradictions. Let's see some examples.

```
#Datatypes conversions
#Variables declarations:
str_example = "5"
print(f"The type of str_example {type(str_example)}")
int_example = 1
print(f"The type of int_example {type(int_example)}")
float_example = 3.4
print(f"The type of float_example {type(float_example)}")
bool_example = True
print(f"The type of bool_example {type(bool_example)}")
another_str = "Alex"
print(f"The type of another_str {type(another_str)}")
#Legal examples:
print("str_example conversions:")
print(f"int conversion: {int(str_example)}, float conversion {float(str_example)}")
print("int_example conversions:")
print(f"string conversion: {str(int_example)}, float conversion {float(int_example)}")
print("float_example conversions:")
print(f"string conversion: {str(float_example)}, int conversion {float(float_example)}")
print("bool_example conversions:")
print(f"int conversion: {int(bool_example)}, float conversion {float(bool_example)}")
```

```
The type of str_example <class 'str'>
The type of int_example <class 'int'>
The type of float_example <class 'float'>
The type of bool_example <class 'bool'>
The type of another_str <class 'str'>
str_example conversions:
int conversion: 5, float conversion 5.0
int_example conversions:
string conversion: 1, float conversion 1.0
float_example conversions:
string conversion: 3.4, int conversion 3.4
bool_example conversions:
int conversion: 1, float conversion 1.0
```

Lists are Indexable

11	7	5	3	2
4	3	2	1	0
1-	2-	3-	4-	5-

```
In [7]: #Lists
        my_list = [2, 3, 5, 7, 11]
        print("Index 0:", my_list[0])
        print("Index 4:", my_list[4])
        print("Index -3:", my_list[-3])
        print("Index 5:", my_list[5])

Index 0: 2
Index 4: 11
Index -3: 5

---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-7-6169f6168904> in <module>
      4 print("Index 4:", my_list[4])
      5 print("Index -3:", my_list[-3])
----> 6 print("Index 5:", my_list[5])

IndexError: list index out of range
```

Lists slicing

	1	2	3	4	5	6	7	8	9	10	
0	1	2	3	4	5	6	7	8	9	10	
-10	-9	-8	-7	-6	-5	4-	3-	2-	1-		

```
#Lists slicing
my_list = [1,2,3,4,5,6,7,8,9,10]
print("Index 1->5:", my_list[1:5])
print("Index 0->-1:", my_list[0:-1])
print("Index beginning->ending, step 2:", my_list[::2])
print("Reverse trick:", my_list[::-1])
print("Legal Index without values:", my_list[3:8:-2])
print("The access doesn't cahnge the list, similar to the strings:", my_list)
```

```
Index 1->5: [2, 3, 4, 5]
Index 0->-1: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Index beginning->ending, step 2: [1, 3, 5, 7, 9]
Reverse trick: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
Legal Index without values: []
The access doesn't cahnge the list, similar to the strings: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Lists – Mutable

- We can update a value in a specific index. This is what makes the List mutable data type.

```
1 #Lists Dynamic
2 students = ["Moshe", 912819379, "Alon", "Zohar", 74472873]
3 print("Before:", students)
4 students[2] = "Alex"
5 print("After:", students)
```

```
Before: ['Moshe', 912819379, 'Alon', 'Zohar', 74472873]
After: ['Moshe', 912819379, 'Alex', 'Zohar', 74472873]
```

- The strings are an example of a data type that is immutable.

```
1 #Strings imuutable
2 student = "Alex Kuznetsov"
3 print("Before:", student)
4 student[0] = "K"
5 print("After:", student)
```

```
Before: Alex Kuznetsov

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-12-e7ead1bf9a54> in <module>
      2 student = "Alex Kuznetsov"
      3 print("Before:", student)
----> 4 student[0] = "K"
      5 print("After:", student)

TypeError: 'str' object does not support item assignment
```

Lists – Mutable continue

Because the lists are mutable we have some methods that can change the list itself, let's see some examples.

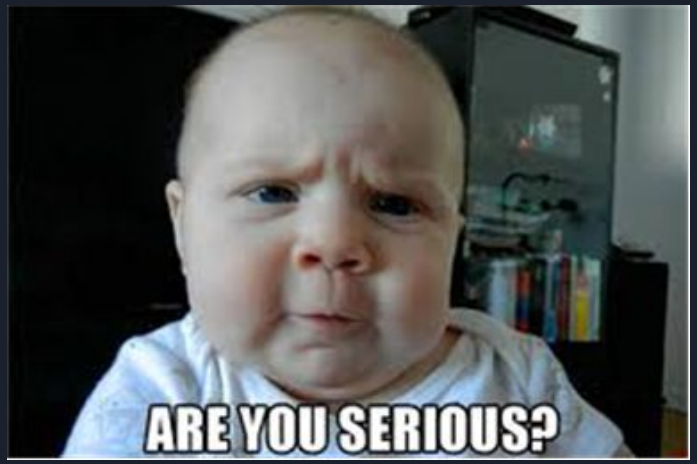
```python
#lists mutable methods
my_list = ['pi', 3.141, True]
"Append, adds a new value to the end of the list"
print("Before:", my_list)
my_list.append("Alex")
print("After:", my_list)
"Remove, removes the first appearence of a value in a list"
my_list.append(True)
print("Before:", my_list)
my_list.remove(True)
print("After:", my_list)
"Pop, returns the last value in the list and removes it from the list"
print("Before:", my_list)
x = my_list.pop()
print("After:", my_list)
print(f"The value returned by pop: {x}")
```

```
Before: ['pi', 3.141, True]
After: ['pi', 3.141, True, 'Alex']
Before: ['pi', 3.141, True, 'Alex', True]
After: ['pi', 3.141, 'Alex', True]
Before: ['pi', 3.141, 'Alex', True]
After: ['pi', 3.141, 'Alex']
The value returned by pop: True
```

Assignments to List Variables

```
1  #Lists behavior in the memory
2  orig_list = [1,2,3]
3  copy_list = orig_list
4  orig_list = [6,7,8,9]
5  print("orig_list value:", orig_list)
6  print("copy_list value:", copy_list)
```

```
orig_list value: [6, 7, 8, 9]
copy_list value: [1, 2, 3]
```



So far - no surprises

```
1  copy_list = orig_list
2  print("orig_list value Before:", orig_list)
3  print("copy_list value Before:", copy_list)
4  orig_list[0] = 1000
5  print("orig_list value After:", orig_list)
6  print("copy_list value After:", copy_list)
```

```
orig_list value Before: [6, 7, 8, 9]
copy_list value Before: [6, 7, 8, 9]
orig_list value After: [1000, 7, 8, 9]
copy_list value After: [1000, 7, 8, 9]
```

Surprise!

Assignments to List Variables

List assignment orig_list = [6,7,8,9] creates:

- a list object, [6,7,8,9]
- a reference from the variable name, orig_list, to this object.

Assignments to List Variables

The assignment copy_list = orig_list does not create a new object, just a new variable name, copy_list, which now refers to the same object.

Assignments to List Variables

We have a function called **id** that can show us the address of a value in the memory let's explain the behavior that we just saw with this function.

```python
#Lists behavior in the memory
orig_list = [1,2,3]
copy_list = orig_list
print("orig_list address before the new assigment:", id(orig_list))
print("copy_list address before the new assigment:", id(copy_list))
orig_list = [6,7,8,9]
print("orig_list address after the new assigment:", id(orig_list))
print("copy_list address after the new assigment:", id(copy_list))
```

```
orig_list address before the new assigment: 2591421532928
copy_list address before the new assigment: 2591421532928
orig_list address after the new assigment: 2591420184256
copy_list address after the new assigment: 2591421532928
```

Exercise

Create a new variable that holds a string of two words, for example, “Alex Kuznetsov”

Create a new variable that holds a list with each word from the string original string as a value correspondingly. For example, in our case the list will be [“Alex”,”Kuznetsov”]. **Hint: To do it you can use the split method on your original string.**

```
#Split example
orig_str = "Alex Kuznetsov"
the_list = orig_str.split()
print("The list:", the_list)
```

```
The list: ['Alex', 'Kuznetsov']
```

Append a new value to your list, whatever you want.

Print the characters from index 1 to index 4 from each word in the list. Do it with the right index access.

Solution

```python
#Split example
orig_str = "Alex Kuznetsov"
the_list = orig_str.split()
print("The list:", the_list)
the_list.append("Python")
print("The list updated:", the_list)
print("The first word:", the_list[0][1:4])
print("The second word:", the_list[1][1:4])
```

```
The list: ['Alex', 'Kuznetsov']
The list updated: ['Alex', 'Kuznetsov', 'Python']
The first word: lex
The second word: uzn
```

Tuples

- A tuple is a collection that is ordered and **unchangeable**.
- A tuple is similar to a list but immutable.
- Syntax: note the parentheses! ()
- We can also create a tuple by just adding a comma to the end of a value.

```
#Tuples example:
tuple_example = ("don't", "worry", "be", "happy")
print("Whole tuple:",tuple_example)
print("Index 0:", tuple_example[0])
print("Index -1:", tuple_example[-1])
print("Index 1->3:", tuple_example[1:3])
```

```
Whole tuple: ("don't", 'worry', 'be', 'happy')
Index 0: don't
Index -1: happy
Index 1->3: ('worry', 'be')
```

```
tuple_without = "Alex", "Kuznetsov"
print("A new tuple:", tuple_without)
print("The type of the value is:", type(tuple_without))
```

```
A new tuple: ('Alex', 'Kuznetsov')
The type of the value is: <class 'tuple'>
```

Tuples continue

```
1 tuple_example[0] = "do"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-6-7ad26b653f7b> in <module>
----> 1 tuple_example[0] = "do"

TypeError: 'tuple' object does not support item assignment
```

No append/extend/remove in Tuples!

What do you think will happen if I will run the following code? And how can you explain such behavior?

```
1 tuple_without = ([1,2,3], [4,5,6])
2 print("Before:", tuple_without)
3 tuple_without[0][1] = 4
4 print("After:", tuple_without)
```

```
Before: ([1, 2, 3], [4, 5, 6])
After: ([1, 4, 3], [4, 5, 6])
```

Sets

- Set is an unordered collection with no duplicate elements.
- Curly braces or the set() function can be used to create sets.

```
1 basket = {"apple", "orange", "apple", "pear", "orange", "bannana"}
2 print("The whole set:", basket)
3 print("The type of the value:", type(basket))
```

```
The whole set: {'pear', 'apple', 'bannana', 'orange'}
The type of the value: <class 'set'>
```

```
1 num_list = [1,1,1,1,1,2,2,2,1,1,2,2,3,3,3,4,1]
2 num_set = set(num_list)
3 print("The num set value:", num_set)
4 print("The num set type:", type(num_set))
```

```
The num set value: {1, 2, 3, 4}
The num set type: <class 'set'>
```

- We can check if a value appears inside a data collection with the “in” command.

```
1 print("orange" in basket)
2 print("kiwi" in basket)
```

```
True
False
```

Sets continue

- Let’s see several methods and functions examples of set structure:

```
#Sets operators
first_set = {1,4}
print("Before:", first_set)
first_set.add(5)
print("After add:", first_set)
first_set.remove(1)
print("After remove:", first_set)
print("The length of the set:", len(first_set))
```

```
Before: {1, 4}
After add: {1, 4, 5}
After remove: {4, 5}
The length of the set: 2
```

- We can unite two sets as well:

```
orig_list = [1, 2, 2, 3, 3, 4, 2, 3, 5, 5]
set_collection = set(orig_list)
second_set = {1, 4, 5}
print(set_collection|second_set)
```

```
{1, 2, 3, 4, 5}
```

Sets operators

- For sets we have some special operators.

```
#Sets operators
num_set_1 = {1, 2, 3, 4, 5, 6}
num_set_2 = {5, 6, 7, 8, 9}
print("Unique values of num_set_2:", num_set_2 - num_set_1)
print("Unique values of num_set_1:", num_set_1 - num_set_2)
print("Combination of both sets:", num_set_1 | num_set_2)
print("Common values for both sets:", num_set_1 & num_set_2)
print("Unique values of num_set_1 and num_set_2:", num_set_1 ^ num_set_2)
```

```
Unique values of num_set_2: {8, 9, 7}
Unique values of num_set_1: {1, 2, 3, 4}
Combination of both sets: {1, 2, 3, 4, 5, 6, 7, 8, 9}
Common values for both sets: {5, 6}
Unique values of num_set_1 and num_set_2: {1, 2, 3, 4, 7, 8, 9}
```

- We can activate the same functionality with some relevant methods as well.

```
#Sets operators
num_set_1 = {1, 2, 3, 4, 5, 6}
num_set_2 = {5, 6, 7, 8, 9}
print("Common values for both sets:", num_set_1.intersection(num_set_2))
print("Combination of both sets:", num_set_1.union(num_set_2))
print("Unique values of num_set_2:", num_set_2.difference(num_set_1))
```

```
Common values for both sets: {5, 6}
Combination of both sets: {1, 2, 3, 4, 5, 6, 7, 8, 9}
Unique values of num_set_2: {8, 9, 7}
```

Dictionaries

- Example: “144” - Map names to phone numbers:

```
#Dictionary
phone_book = {"Alex Kuznetsov" : "0527389001", "Stan March" : "0912013803", "Kyle Broflovski" : "12168326329"}
print("Dictionary example:", phone_book)
```

```
Dictionary example: {'Alex Kuznetsov': '0527389001', 'Stan March': '0912013803', 'Kyle Broflovski': '12168326329'}
```

- We can access dictionary value by its key.

```
print("The value of Alex Kuznetsov:", phone_book["Alex Kuznetsov"])
```

```
The value of Alex Kuznetsov: 0527389001
```

- Add a new key and value to a dictionary.

```
#Add a new key and value to a dictionary
phone_book["Bruce Wayne"] = "2839289271"
print("The updated dictionary:", phone_book)
```

```
The updated dictionary: {'Alex Kuznetsov': '0527389001', 'Stan March': '0912013803', 'Kyle Broflovski': '12168326329', 'Bruce W
ayne': '2839289271'}
```

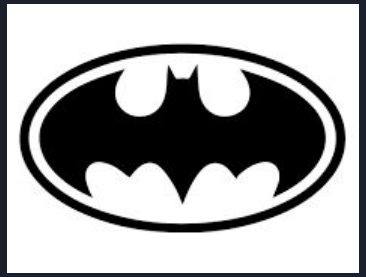
Dictionaries continue

What happens when we add a key that already exists?

```
#Update a value of an existing key in a dictionary
phone_book["Bruce Wayne"] = "11111111111"
print("The updated dictionary:", phone_book)
```

```
The updated dictionary: {'Alex Kuznetsov': '0527389001', 'Stan March': '0912013803', 'Kyle Broflovski': '12168326329', 'Bruce W
ayne': '11111111111'}
```

Bruce's phone was previously 2093029018 and now changed to 1111111111



Dictionaries continue

Idea: Add address to the key, Basically we want to use data collection as a key.

```
1 phone_book = {["Bruce Wayne", "Gotham"] : "1111111111"}
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-33-e209dad4d0f3> in <module>
----> 1 phone_book = {["Bruce Wayne", "Gotham"] : "1111111111"}

TypeError: unhashable type: 'list'
```

What's the problem?

Keys are **unique** and **immutable**.

Solution?

Dictionaries continue

Fix: use tuples as keys!

```
1 phone_book = {("Bruce Wayne", "Gotham") : "1111111111"}
2 print(phone_book)
3 print(phone_book[("Bruce Wayne", "Gotham")])
```

```
{('Bruce Wayne', 'Gotham'): '1111111111'}
1111111111
```

Dictionary Methods

```python
#Dictionary methods
phone_book = {"Alex Kuznetsov" : "0527389001", "Stan March" : "0912013803", "Kyle Broflovski" : "12168326329"}
#Get method
print("Existing key:", phone_book.get("Alex Kuznetsov", 0))
print("Not existing key:", phone_book.get("Tom Kuznetsov", 0))
#Items method
print("The items of the dictionary:", phone_book.items())
print("The type of the items output:", type(phone_book.items()))
#Keys method
print("The keys of the dictionary:", phone_book.keys())
print("The type of the items output:", type(phone_book.keys()))
#Values method
print("The keys of the dictionary:", phone_book.values())
print("The type of the items output:", type(phone_book.values()))
#Pop method
print("Existing key:", phone_book.pop("Alex Kuznetsov", 0))
print("Not existing key:", phone_book.pop("Tom Kuznetsov", 0))
print("The dictionary after pop execution:", phone_book)
#Update method
print("Before:", phone_book)
phone_book.update({"Tom Yelnikoff" : 2362863})
print("After:", phone_book)
```

```
Existing key: 0527389001
Not existing key: 0
The items of the dictionary: dict_items([('Alex Kuznetsov', '0527389001'), ('Stan March', '0912013803'), ('Kyle Broflovski', '1
2168326329')])
The type of the items output: <class 'dict_items'>
The keys of the dictionary: dict_keys(['Alex Kuznetsov', 'Stan March', 'Kyle Broflovski'])
The type of the items output: <class 'dict_keys'>
The keys of the dictionary: dict_values(['0527389001', '0912013803', '12168326329'])
The type of the items output: <class 'dict_values'>
Existing key: 0527389001
Not existing key: 0
The dictionary after pop execution: {'Stan March': '0912013803', 'Kyle Broflovski': '12168326329'}
Before: {'Stan March': '0912013803', 'Kyle Broflovski': '12168326329'}
After: {'Stan March': '0912013803', 'Kyle Broflovski': '12168326329', 'Tom Yelnikoff': 2362863}
```

Exercise

- Create a set that will hold the values of the first dictionary and a set that will hold the values of the second dictionary.
- Create a united set of both sets from the previous section.
- Create a set that will hold the unique values of the first set and the second set. Use the relevant operators between the sets.
- Add a new string key to the first dictionary with the value of the unique values set that you created in the previous section.
- Print all variables that you used.

Solution

```python
dict1_sol = {'j': 2, 'a': 4, 's': 3, 'k': 2, 'd': 3, 'h': 3, 'l': 2}
dict2_sol = {'j': 3, 'a': 3, 'b': 4, 'c': 2, 'd': 3, 'e': 5, 'l': 3}
set1_sol = set(dict1_sol.values())
print("Set of dict1_sol values:", set1_sol)
set2_sol = set(dict2_sol.values())
print("Set of dict2_sol values:", set2_sol)
united_set = set1_sol | set2_sol
print("United set:", united_set)
unique_set = set1_sol ^ set2_sol
print("Unique set:", unique_set)
dict1_sol["Solution"] = unique_set
print("The final dictionary:", dict1_sol)
```

```
Set of dict1_sol values: {2, 3, 4}
Set of dict2_sol values: {2, 3, 4, 5}
United set: {2, 3, 4, 5}
Unique set: {5}
The final dictionary: {'j': 2, 'a': 4, 's': 3, 'k': 2, 'd': 3, 'h': 3, 'l': 2, 'Solution': {5}}
```

Thank you for your time